ABSTRACT

Retail demand forecasting plays a crucial role in inventory optimization by enabling businesses to maintain optimal stock levels while minimizing overstocking and stockouts. This project addresses the problem of demand prediction by leveraging machine learning techniques on a retail dataset comprising 2,000 records and 58 features.

The study explores both regression and classification approaches. For classification, the target variable *Daily_Units_Sold* is transformed categories, enabling multi-class demand prediction. A comprehensive data preprocessing pipeline was implemented, including missing value imputation, feature engineering, dimensionality reduction using PCA, and handling of high-cardinality features.

To effectively utilize textual and categorical data, multiple feature representation techniques were employed, including categorical encoding, TF-IDF, Word2Vec, and pre-trained GloVe embeddings. Three types of models were developed and evaluated: a baseline Multinomial Logistic Regression model, a custom deep learning model, and a transformer-based model.

Experimental results indicate that traditional machine learning models perform competitively, often matching or outperforming more complex deep learning approaches depending on the feature representation used. The project also demonstrates practical applicability by deploying the final model using FastAPI, enabling real-time demand prediction.

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Full Form
ML	Machine Learning
DL	Deep Learning
NLP	Natural Language Processing
PCA	Principal Component Analysis
TF-IDF	Term Frequency – Inverse Document Frequency
GloVe	Global Vectors for Word Representation
RF	Random Forest
LR	Logistic Regression
DNN	Deep Neural Network
BN	Batch Normalization
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
R^2	Coefficient of Determination
F1	F1-Score
MAPE	Mean Absolute Percentage Error
SKU	Stock Keeping Unit
LSTM	Long Short-Term Memory
XGBoost	Extreme Gradient Boosting
BITS	Birla Institute of Technology and Science
API	Application Programming Interface

TABLE OF CONTENTS

Chapter 1: Introduction 1

1.1 Problem Statement 1

1.2 Objectives of the project 1

1.3 Scope of the project 1

Chapter 2: Literature Review 2

2.1 Demand Forecasting with Machine Learning: A Data-Driven Strategy for Market Prediction 2

2.2 Smart Retail Using Machine Learning for Demand Forecasting and Inventory Optimization 3

2.3 Optimizing Retail Operations with Big Data-Driven Insights: From Inventory Management to Personalized Marketing 3

2.4 Summary of Reviewed Works 4

Chapter 3: Methodology 5

3.1 ML Process Flow / Solution Architecture 5

3.2 Dataset Description 6

3.3 Overview from Data Profiling report 7

3.4 Resources 8

3.5 Data Challenges and Risks 9

Chapter 4: Data Preprocessing and Feature Engineering 10

4.1 Data Loading and Initial Inspection 10

4.2 Data Cleaning 10

4.3 Feature / Target Split 10

4.4 Data Preprocessing Pipeline 11

4.5 Feature - Feature Correlation Analysis 12

4.6 Feature - Target Correlation Analysis 13

4.7 Leakage Detection 13

4.8 PCA-based Dimensionality Reduction 14

4.9 Baseline Encoding (Label Encoding) 14

4.10 TF-IDF Vectorization 15

4.11 Word2Vec Vectorization 15

4.12 GloVe Vectorization 16

Chapter 5: Machine Learning Modelling - Classification 17

5.1 Logistic Regression 17

5.2 Diamond Deep Neural Network 18

5.3 Hybrid Transformer Model 19

Chapter 6: Machine Learning Modelling - Regression 20

6.1 Random Forest Regressor 20

6.2 Diamond Deep Learning Regression Model 20

6.3 Hybrid Transformer Regression Model 21

Chapter 7: Results and Interpretation 23

7.1 Metric Justification 23

7.2 Final Model Selection 24

7.3 Model Deployment and Inference Pipeline 24

Chapter 8: Code Implementation and Execution Results 26

8.1 Data Cleaning and Preprocessing 26

8.2 Feature and Target Definition 26

8.3 Feature Preprocessing and Scaling 26

8.4 Dimensionality Reduction using PCA 27

8.5 Categorical Feature Encoding 27

8.6 TF-IDF Vectorization 27

8.7 Word2Vec Embedding 28

8.8 GloVe Embedding 28

8.9 Model Training – Logistic Regression (Classification Baseline) 28

8.10 Model Training – Deep Learning Model (Classification) 29

8.11 Model Training – Transformer Model (Classification) 29

8.12 Model Training – Random Forest (Regression Baseline) 30

8.13 Model Training – Deep Learning Model (Regression) 31

8.14 Model Training – Transformer Model (Regression) 32

8.15 Fast API 33

Chapter 9: Conclusions and Future Work 35

9.1 Conclusion 35

9.2 Directions for Future Work 36

References 37

LIST OF TABLES

Table 1: Reference summary 4

Table 2: Data description..... 6

Table 3: Classification Bins.....17

Table 4: Deep Learning model 21

Table 5: Evaluation Metrics 23

LIST OF FIGURES

Figure 1: ML Process Flow 5
Figure 2: Architecture 5
Figure 3: Data Profiler 7
Figure 4: Data Profiler – Correlation Analysis 7
Figure 5: Data Profiler - Missing data 8
Figure 6: Feature vs Feature Correlation 12
Figure 7: Feature vs Target Correlation 13
Figure 8: Custom Deep Learning model 18
Figure 9: FastAPI architecture 25
Figure 10: Data Preprocessing output 26
Figure 11: Feature Split output 26
Figure 12: Feature preparation output 27
Figure 13: PCA output 27
Figure 14: Encoding output 27
Figure 15: TF - IDF output 27
Figure 16: Word2Vec output 28
Figure 17: GloVe output 28
Figure 18: Logistic Regression output 28
Figure 19: Deep Learning output 29
Figure 20: Transformer output 29
Figure 21: Regression output 30
Figure 22: Deep Learning output 31
Figure 23: Transformer output 32
Figure 24: Transformer output 32
Figure 25: FastAPI default UI 33

Figure 26: FastAPI parameters view.....33

Figure 27: FastAPI predict view.....34

Figure 28: FastAPI responses 34

Chapter 1: Introduction

1.1 Problem Statement

Retail businesses require accurate demand forecasting to maintain profitability and customer satisfaction. Overstocking increases holding costs and risks obsolescence, while stockouts lead to lost sales and reduced customer loyalty. Traditional methods like moving averages, exponential smoothing, and ARIMA rely on historical data and assume linear patterns, making them inadequate for modern retail environments.

Demand today is influenced by multiple factors such as promotions, seasonality, pricing, social sentiment, online traffic, weather, and store footfall. This project addresses the limitation by using multi-class machine learning classification to predict daily demand categories, enabling more accurate and data-driven inventory decisions.

1.2 Objectives of the project

The primary objectives of this project are:

- To perform exploratory data analysis (EDA) and build a robust preprocessing pipeline, including feature extraction, missing value handling, and data cleaning.
- To apply dimensionality reduction (PCA) and evaluate multiple encoding techniques such as Label Encoding, TF-IDF, Word2Vec, and GloVe.
- To train and compare Logistic Regression, Deep Neural Network, and Hybrid Transformer models.
- To identify the best-performing model and interpret results for effective retail decision-making.

1.3 Scope of the project

This project covers the complete machine learning lifecycle, from data preprocessing to model evaluation and interpretation, using a structured retail dataset. The multi class demand classification output provides actionable insights for inventory planning and replenishment. The solution is designed to be scalable and adaptable across different retail environments, forming a foundation for real-world deployment using live transactional data.

Chapter 2: Literature Review

Demand forecasting plays a critical role in supply chain management by enabling organizations to optimize inventory levels, reduce costs, and improve customer satisfaction. [1] With the growing complexity of modern supply chains, traditional statistical methods are increasingly being supplemented or replaced by artificial intelligence (AI) and machine learning (ML) techniques that offer improved accuracy and adaptability [2]. Recent advancements in AI-driven forecasting methods, including the use of deep learning and hybrid models, have demonstrated significant potential in capturing complex demand patterns and enhancing decision-making processes in dynamic environments [3].

The following literature review is grounded in three IEEE conference papers directly relevant to this project, supplemented by broader context from the field of machine learning applied to retail demand forecasting

2.1 Demand Forecasting with Machine Learning: A Data-Driven Strategy for Market Prediction

This paper presents a machine learning–based demand forecasting system for predicting daily item-store sales using historical data. It frames retail forecasting as a time-series prediction problem and highlights the limitations of traditional models like ARIMA in capturing non-linear patterns influenced by seasonality, promotions, and store-specific trends. The study evaluates Random Forest Regressor, Gradient Boosting, and Linear Regression, finding Random Forest to perform best due to its ability to handle complex, high-dimensional data and resist overfitting. Model performance is assessed using MSE, RMSE, and R^2, with results showing clear seasonal demand peaks in June–July and lows in January–March.

The paper strongly informs this project's methodology, particularly in adopting Random Forest for regression and feature importance analysis. It emphasizes temporal feature engineering by extracting components such as day, month, year, and holiday indicators from date fields—an approach mirrored in this project's preprocessing pipeline. Additionally, its analysis of outliers (94.7% normal, 5.3% anomalous) supports the use of an outlier-aware imputation strategy, applying median and mode values to minimize distortion from anomalies. [4]

2.2 Smart Retail Using Machine Learning for Demand Forecasting and Inventory Optimization

This paper proposes a comprehensive machine learning framework for joint demand forecasting and inventory optimization, addressing the shortcomings of traditional methods like moving averages and exponential smoothing in handling complex, real-time retail demand. The authors employ LSTM networks and XGBoost as their primary models—LSTM for capturing long-term temporal dependencies and XGBoost for efficient, high-accuracy ensemble learning. Using MAPE as the evaluation metric, the framework achieves strong forecasting performance (6.8% short-term, 7.2% medium-term, 8.4% long-term), significantly outperforming traditional approaches, which show much higher error rates.

Beyond forecasting, the system demonstrates substantial improvements in inventory management, achieving a higher inventory turnover ratio (6.3) and significantly reducing stockout and overstock incidents compared to benchmark systems. It also records higher user satisfaction across usability and accuracy. These findings influence this project in two key ways: first, by supporting a multi-model evaluation strategy that includes classical, ensemble, and deep learning approaches; and second, by reinforcing the importance of actionable outputs, where this project's demand categorization serves a similar role to inventory optimization by providing clear reorder signals. [5]

2.3 Optimizing Retail Operations with Big Data-Driven Insights: From Inventory Management to Personalized Marketing

This paper explores the transformative impact of big data analytics on retail operations, particularly in inventory management, supply chain efficiency, and personalized marketing. The results demonstrate significant improvements, including reductions in stockout rates (from 12.5% to 4.2%) and overstock rates (from 18.7% to 6.5%), along with an increase in inventory turnover ratio from 5.3 to 8.7. Additionally, holding costs decreased by 38%. Supply chain performance also improved markedly, with order fulfillment time reduced from 48 to 24 hours, transportation costs lowered by 25%, delivery accuracy increased to 95%, and supplier lead times shortened by 40%. Personalized marketing outcomes showed strong gains, with conversion rates rising from 8.3% to 15.7% and customer lifetime value increasing by 44%.

The methodology integrates multiple forecasting models (ARIMA, LSTM, Prophet) alongside clustering techniques like K-means and hierarchical clustering, leveraging data from

diverse sources such as POS systems, online platforms, loyalty programs, and IoT sensors. This directly supports the design of this project's feature-rich dataset, which incorporates transactional, behavioral, and external factors like footfall, sentiment, pricing, promotions, and weather. The paper reinforces the business value of accurate demand forecasting and provides a strong benchmark for operational impact, validating the approach of using multi-dimensional data and demand classification to drive inventory and decision-making efficiency. [6]

2.4 Summary of Reviewed Works

Paper	Method	Key Finding
Demand Forecasting with Machine Learning: A Data-Driven Strategy for Market Prediction	Random Forest + Gradient Boosting	Seasonal peaks identified; RF outperforms LR for non-linear sales patterns
Smart Retail Using Machine Learning for Demand Forecasting and Inventory Optimization	LSTM + XGBoost	MAPE 6.8% short-term; Inventory turnover 6.3; 18 stockout incidents
Optimizing Retail Operations with Big Data-Driven Insights: From Inventory Management to Personalized Marketing	ARIMA + LSTM + Prophet + K - Means	66.4% stockout reduction; 64.2% turnover improvement; 89.2% conversion gain

Table 1: Reference summary

Chapter 3: Methodology

3.1 ML Process Flow / Solution Architecture

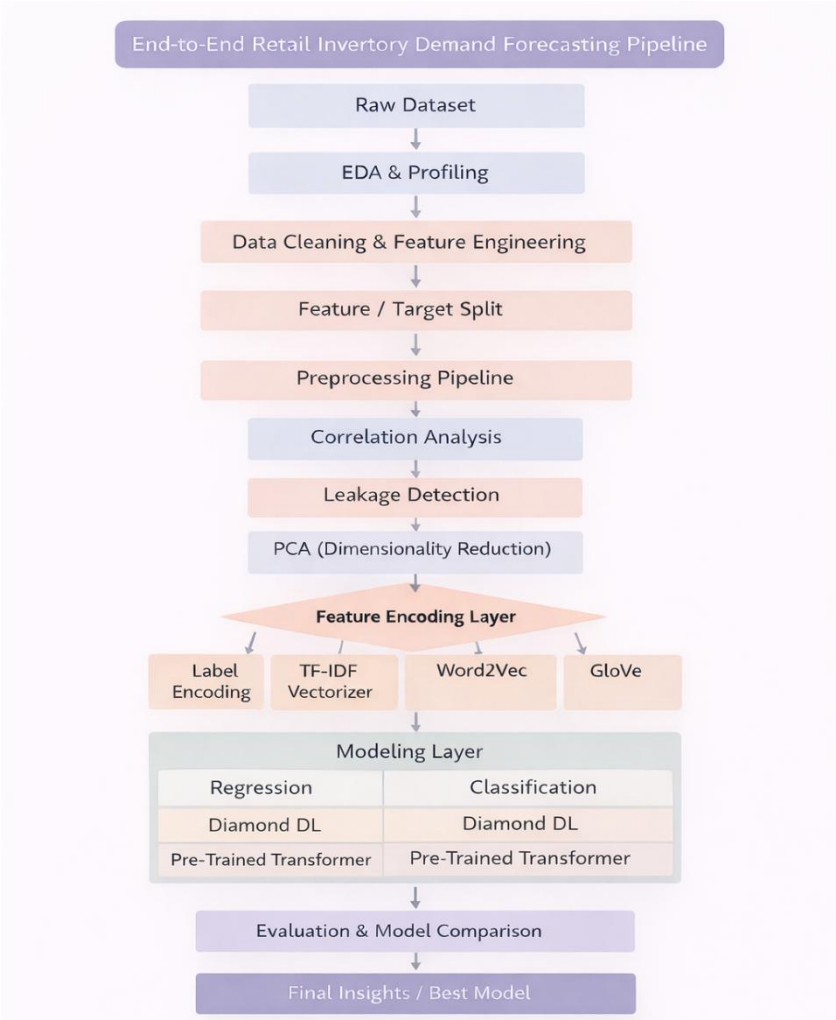

Figure 1: ML Process Flow

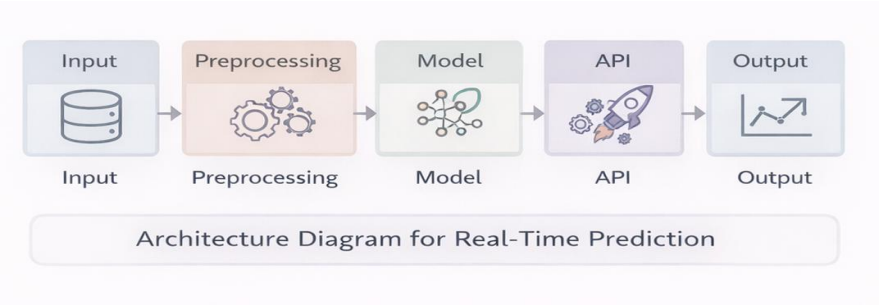

Figure 2: Architecture

3.2 Dataset Description

The dataset (master_retail_dataset_v2.csv) contains 2,000 daily retail records with 58 features across the following categories:

Feature Category	Representative Columns	Count
Temporal	Date, DayOfWeek, Month, Year, Is_Weekend, Is_Holiday	6
Product	Product_ID, Product_Name, Category, Sub_Category, Brand_Name, Brand_Tier, Shelf_Life_Days	7+
Pricing	Base_Price, Discount_Percentage, Current_Price, Competitor_Price, Unit_Cost	5
Store	Store_ID, Store_Type, No_of_Checkout_Counters, Avg_Billing_Time_min	4
External	Footfall_Index, Avg_Temperature, Rainfall_mm, Social_Media_Sentiment, Local_Population_Density	5
Supply Chain	Lead_Time_Days, Safety_Stock_Level, Stock_On_Hand, Backorder_Flag	4
Promotional	Promotion_Type, Promotional_Campaign_Flag, Competitor_Promotion_Flag, Festival_Name, Festival_Type	5+
Customer Behaviour	No_of_Customer_Purchases, Loyalty_Program_Usage_Count, Website_Visits, App_Traffic_Index	4+
Target	Daily_Units_Sold	1

Table 2: Data description

3.3 Overview from Data Profiling report

Overview | Alerts 41 | Reproduction

Dataset statistics

Number of variables	58
Number of observations	2000
Missing cells	4443
Missing cells (%)	3.8%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	2.1 MiB
Average record size in memory	1.1 KiB

Variable types

DateTime	2
Numeric	37
Categorical	14
Text	1
Boolean	4

Figure 3: Data Profiler

Overview | Alerts 41 | Reproduction

Alerts

Year has constant value "2023"	Constant
App_Traffic_Index is highly overall correlated with Daily_Units_Sold and 2 other fields	High correlation
Avg_Basket_Size is highly overall correlated with Loyalty_Program_Usage_Count and 1 other fields	High correlation
Backorder_Flag is highly overall correlated with Stock_On_Hand	High correlation
Base_Price is highly overall correlated with Competitor_Price and 4 other fields	High correlation
Brand_Name is highly overall correlated with Category and 3 other fields	High correlation
Category is highly overall correlated with Brand_Name and 2 other fields	High correlation
Competitor_Price is highly overall correlated with Base_Price and 4 other fields	High correlation
Current_Price is highly overall correlated with Base_Price and 4 other fields	High correlation
Daily_Units_Sold is highly overall correlated with App_Traffic_Index and 7 other fields	High correlation
DayOfWeek is highly overall correlated with Festival_Name and 2 other fields	High correlation
Discount_Percentage is highly overall correlated with Marketing_Spend and 1 other fields	High correlation
Festival_Name is highly overall correlated with DayOfWeek and 5 other fields	High correlation
Festival_Type is highly overall correlated with DayOfWeek and 3 other fields	High correlation
In_Store_Sales_Units is highly overall correlated with Base_Price and 4 other fields	High correlation
Is_Weekend is highly overall correlated with DayOfWeek and 1 other fields	High correlation
Local_Population_Density is highly overall correlated with Store_Type	High correlation
Loyalty_Program_Usage_Count is highly overall correlated with Avg_Basket_Size and 1 other fields	High correlation
Marketing_Spend is highly overall correlated with Discount_Percentage and 2 other fields	High correlation
Month is highly overall correlated with Festival_Name and 2 other fields	High correlation
No_of_Customer_Purchases is highly overall correlated with Avg_Basket_Size and 1 other fields	High correlation
Online_Sales_Units is highly overall correlated with App_Traffic_Index and 2 other fields	High correlation
Payday_Flag is highly overall correlated with Festival_Name	High correlation
Promotion_Type is highly overall correlated with Promotional_Campaign_Flag	High correlation
Promotional_Campaign_Flag is highly overall correlated with Discount_Percentage and 2 other fields	High correlation
School_Vacation_Flag is highly overall correlated with Festival_Name and 1 other fields	High correlation
Seasonal_Product_Flag is highly overall correlated with Brand_Name and 1 other fields	High correlation
Shelf_Life_Days is highly overall correlated with Brand_Name and 2 other fields	High correlation
Stock_On_Hand is highly overall correlated with Backorder_Flag	High correlation
Store_Type is highly overall correlated with Local_Population_Density	High correlation

Figure 4: Data Profiler – Correlation Analysis

Sub_Category is highly overall correlated with Brand_Name and 3 other fields	High correlation
Unit_Cost is highly overall correlated with Base_Price and 4 other fields	High correlation
Website_Visits is highly overall correlated with App_Traffic_Index and 2 other fields	High correlation
Is_Holiday is highly imbalanced (55.4%)	Imbalance
Local_Event_Flag is highly imbalanced (72.4%)	Imbalance
Festival_Name has 1945 (97.2%) missing values	Missing
Festival_Type has 1945 (97.2%) missing values	Missing
Promotion_Type has 553 (27.7%) missing values	Missing
DayOfWeek has 291 (14.5%) zeros	Zeros
Discount_Percentage has 23 (1.1%) zeros	Zeros
Supplier_Delay_Days has 454 (22.7%) zeros	Zeros

Figure 5: Data Profiler - Missing data

3.4 Resources

Hardware:

Personal computer running Windows (Anaconda environment); GPU support via WSL2 or CUDA-enabled setup recommended for training deep learning models.

Software and Libraries:

- **Python 3.x with Jupyter Notebook** – core development environment
- **pandas, numpy** – data manipulation, preprocessing, and feature engineering
- **matplotlib, seaborn** – data visualization (trend analysis, correlation plots, and performance graphs)
- **scikit-learn** – preprocessing, train-test splitting, evaluation metrics, and baseline models
- **TensorFlow / Keras** – development of deep learning forecasting models
- **XGBoost** – gradient boosting model for residual learning and prediction refinement
- **Hugging Face Transformers** – implementation and usage of Chronos pretrained time-series model
- **FastAPI** – deployment of deep learning model as a REST API for real-time or batch inference
- **uvicorn** – ASGI server for running FastAPI applications

3.5 Data Challenges and Risks

- **Data Leakage:** *In_Store_Sales_Units* (correlation: **0.907**) and *Online_Sales_Units* (**0.564**) were identified as leakage-prone features due to their strong direct relationship with the target variable. Additionally, *Current_Price* (**0.795**), *Base_Price* (**0.749**), *Competitor_Price* (**0.746**), and *Unit_Cost* (**0.719**) exhibited high correlation (>0.70) and were treated as potential leakage candidates. These features were handled through **PCA-based dimensionality reduction**.
- **High-Cardinality Identifier Columns:** *UPC_EAN* and *Product_ID* were identified as near-unique identifier columns (with >80% unique values) and were removed from the feature set to prevent **spurious memorization** by the models.
- **Class Balance:** Quantile-based binning using qcut (with duplicates='drop') was applied to transform the target variable into five approximately balanced demand classes (~400 samples per class), thereby mitigating class imbalance during classification.
- **Text Column Heterogeneity:** Columns such as *Festival_Name* and *Product_Name* contain high semantic value but relatively low statistical frequency per unique entry. To address this, specialized preprocessing was applied, including:
  - Ensuring a minimum of two unique values for TF-IDF vectorization
  - Using a custom tokenization pattern to effectively handle alphanumeric retail identifiers
- **Computational Constraints:** Advanced modeling components such as GloVe embeddings (100-dimensional), Diamond Deep Neural Network training (20 epochs), and DistilBERT fine-tuning are computationally intensive. Additionally, TensorFlow GPU support is not natively available on Windows (version ≥2.11); therefore, **WSL2 is recommended** for efficient training in production environments.

Chapter 4: Data Preprocessing and Feature Engineering

4.1 Data Loading and Initial Inspection

The dataset was loaded using pandas read_csv. The initial inspection revealed a shape of 2,000 rows × 58 columns with mixed data types. No duplicate rows were found. Nine object-type columns remained after binary column encoding: Category, Store_Type, Festival_Name, Festival_Type, Product_Name, Sub_Category, Brand_Name, Brand_Tier, and Promotion_Type.

4.2 Data Cleaning

A clean dataset (df_clean) was created using an automated preprocessing pipeline to handle inconsistencies, missing values, and feature transformations.

Date-related columns were dynamically detected and converted to datetime format. Additional features such as day and month were extracted to capture temporal patterns. Missing values were handled using a type-based strategy: numerical features were imputed with the median, while categorical features used the mode or a constant value ("missing") when required.

Binary categorical features (e.g., Y/N) were converted into numeric format (1/0) for model compatibility. High-cardinality numerical features were removed using a 95% uniqueness threshold to reduce noise and overfitting. After preprocessing, the dataset retained its shape of 2,000 × 58 with no missing values. Remaining categorical features were preserved for further encoding and vectorization in later stages.

4.3 Feature / Target Split

After data cleaning, the dataset was split into input features and the target variable for model training. The target variable was defined as **Daily_Units_Sold**, and its presence was validated before proceeding.

The dataset was then divided into input features (**X**) and target variable (**y**), where X contains all columns except the target. This ensures the model learns patterns from features to predict sales accurately.

Feature types were identified dynamically based on data types. Numerical features (int64, float64) included variables such as DayOfWeek, Month, Year, Discount_Percentage, Footfall_Index, and

Avg_Temperature. Categorical features (object type) included Category, Store_Type, Sub_Category, Brand_Name, Brand_Tier, and Promotion_Type.

Text features were detected using a heuristic based on string length. Columns with longer average text length were classified as text features, identifying Festival_Name and Product_Name for NLP-based processing.

Finally, categorical features were refined after removing text columns. This structured separation into numerical, categorical, and text features enables appropriate preprocessing in later stages.

4.4 Data Preprocessing Pipeline

A refined dataset was created using a systematic preprocessing pipeline to improve data quality and model readiness. ID-like columns were identified using a uniqueness threshold of 80%. Columns exceeding this threshold were treated as identifiers and removed to reduce noise and prevent data leakage.

Missing values were handled using a type-based approach. Numerical features were imputed with the median, categorical features with the constant value “missing”, and text features with empty strings to support vectorization.

Features were grouped into three categories: numerical, categorical, and text. Numerical features included variables like DayOfWeek, Month, Year, Discount_Percentage, Footfall_Index, and Avg_Temperature. Categorical features included Category, Store_Type, Sub_Category, Brand_Name, Brand_Tier, and Promotion_Type. Text features included Festival_Name and Product_Name for NLP-based processing.

All numerical features were standardized using StandardScaler to ensure uniform scaling and improve model performance. After preprocessing, the dataset had a final shape of **2,000 × 56**, with all missing values handled and features properly structured for modeling.

Some ID-like columns such as Product_ID, Store_ID, and UPC_EAN were still present. This indicates that their uniqueness did not exceed the threshold or requires further validation. It highlights the need to combine automated methods with domain knowledge for better feature selection.

4.5 Feature - Feature Correlation Analysis

A correlation analysis was performed on all numerical features using a Seaborn heatmap (50 × 50, coolwarm palette). The results showed strong correlation clusters, especially among pricing features (Base_Price, Current_Price, Competitor_Price, Unit_Cost) and digital engagement features (Website_Visits, App_Traffic_Index, Online_Sales_Units).

These patterns indicate high multicollinearity, which can reduce model performance and interpretability. To address this, PCA was applied to reduce correlated features into a smaller set of representative components.

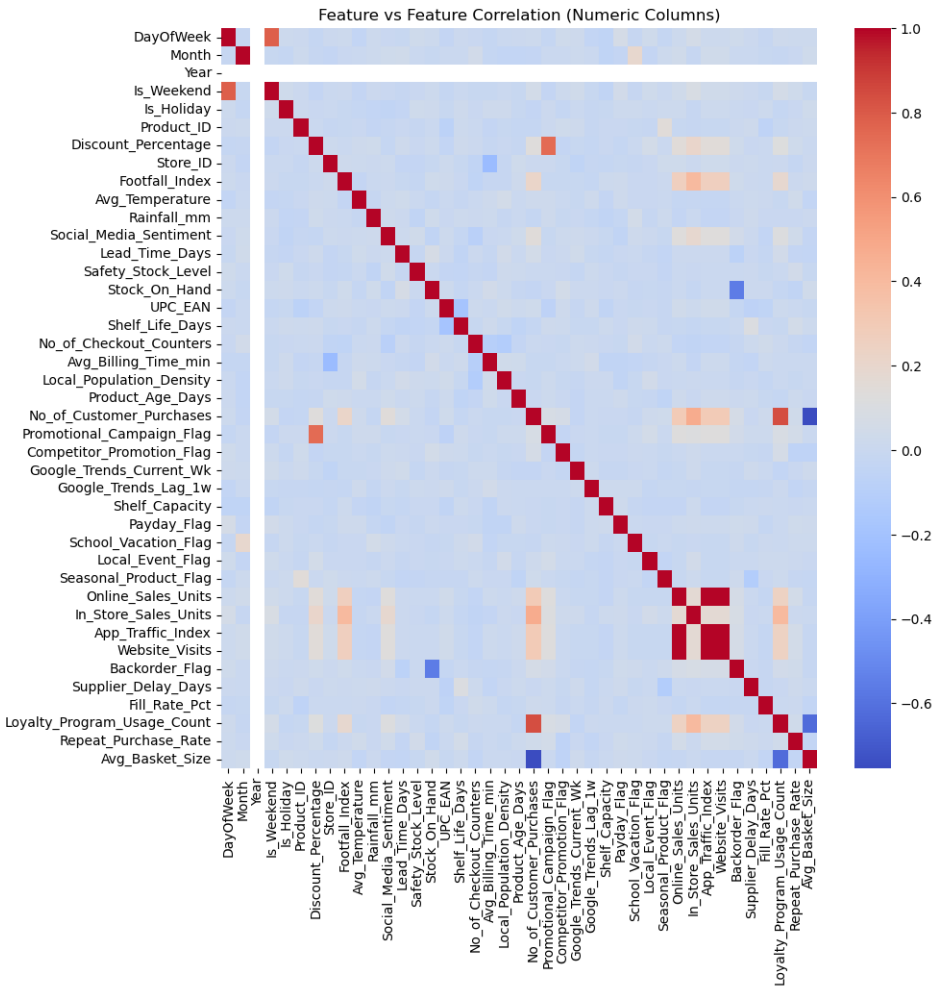

Figure 6: Feature vs Feature Correlation

4.6 Feature - Target Correlation Analysis

To understand the relationship between input features and the target variable (**Daily_Units_Sold**), a bar chart of the top 15 correlated features was generated.

The analysis showed a strong correlation for **In_Store_Sales_Units (0.907)**, indicating potential data leakage. Moderate correlations were observed for online traffic features (~0.56) and **No_of_Customer_Purchases (0.526)**. Among reliable predictors, **Footfall_Index (0.444)** and **Loyalty_Program_Usage_Count (0.437)** showed the highest correlation.

These results helped identify leakage-prone variables, select meaningful predictors, and guide feature engineering and model design.

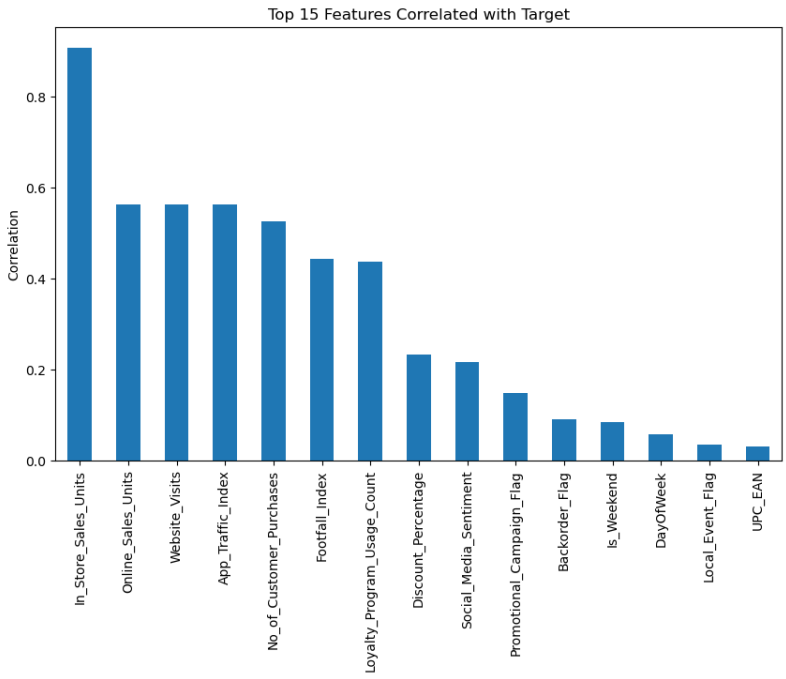

Figure 7: Feature vs Target Correlation

4.7 Leakage Detection

Features with an very high correlation with the target were identified as potential statistical leakage. Additionally, columns containing “sales_units” were flagged as conceptual leakage since they represent post-outcome information not available at prediction time.

This analysis identified **In_Store_Sales_Units** and **Online_Sales_Units** as the main leakage features. Pricing-related variables with moderately high correlation (0.72–0.80) were retained and

handled using PCA-based compression to reduce multicollinearity while preserving useful information.

4.8 PCA-based Dimensionality Reduction

PCA was applied to reduce redundancy and improve model efficiency by targeting highly correlated numerical features. Highly correlated feature pairs were identified from the correlation matrix using a threshold greater than 0.9. Only the upper triangular matrix was used to avoid duplicate comparisons. To ensure meaningful selection, a variance threshold of 0.01 was also applied, removing low-variance features.

Based on these criteria, features such as Online_Sales_Units, Month, Website_Visits, Date_month, and App_Traffic_Index were selected for PCA. These features were standardized using StandardScaler before transformation.

PCA was then applied, retaining 97% of the total variance and producing **2 principal components**. The explained variance ratios were 0.6014 and 0.3986, with a cumulative variance of approximately 0.9999.

The original correlated features were replaced with the PCA components (PCA_HighCorr_1, PCA_HighCorr_2). After transformation, the dataset size was reduced to **(2000, 54)** while preserving most of the information.

4.9 Baseline Encoding (Label Encoding)

After dimensionality reduction, categorical features were encoded into numerical format for machine learning compatibility. A copy of the PCA-transformed dataset (**df_encoded**) was created to preserve the original data.

Categorical columns were identified dynamically using object data type. Features with longer average string length (>15 characters) were classified as text features for later NLP processing, while the remaining object-type columns were treated as categorical features.

The categorical features (**Category, Store_Type, Festival_Type, Sub_Category, Brand_Name, Brand_Tier, Promotion_Type**) were encoded using Label Encoding. Each unique value was mapped to an integer, and separate encoders were stored for consistency during inference.

Text features (**Festival_Name, Product_Name**) were retained for future vectorization. After encoding, the dataset shape remained **(2000, 54)**, as encoding did not change dimensionality but converted values into numerical form.

4.10 TF-IDF Vectorization

To convert textual and categorical string features into numerical form, TF-IDF (Term Frequency–Inverse Document Frequency) vectorization was applied. A separate copy of the dataset (**df_tfidf**) was created to preserve the original data.

All object-type columns were dynamically identified as text features, including Category, Store_Type, Festival_Name, Festival_Type, Product_Name, Sub_Category, Brand_Name, Brand_Tier, and Promotion_Type. This allowed consistent handling of both categorical and descriptive text data.

TF-IDF was applied separately to each column with more than one unique value. Each column was transformed into a sparse matrix capturing term importance, which was then converted into a dense DataFrame with column-specific prefixes.

The resulting TF-IDF features were concatenated with the original dataset, significantly increasing dimensionality. The final dataset shape became **(2000, 192)**. While this increases sparsity and computational cost, it helps capture detailed textual patterns for improved model learning.

4.11 Word2Vec Vectorization

To capture semantic relationships in textual data, Word2Vec embeddings were generated for all text-based features. A separate copy of the dataset (**df_w2v**) was created to preserve the original PCA-transformed data.

All object-type columns were identified as text features and processed individually. Each text entry was tokenized into words to prepare it for embedding generation. Columns with empty values or a single unique entry were excluded to ensure meaningful representations.

For each eligible column, a separate Word2Vec model was trained with an embedding size of 50, window size of 5, minimum word frequency of 1, and a fixed seed for reproducibility. This model learns contextual relationships between words based on their usage in the dataset.

Since Word2Vec produces word-level vectors, row-level representations were obtained by averaging all word embeddings in each record. If no valid tokens were present, a zero vector was assigned.

The resulting embeddings were prefixed by column name and concatenated with the dataset. After transformation, the dataset size increased to **(2000, 504)**. While this improves semantic understanding, it also increases dimensionality and computational cost.

4.12 GloVe Vectorization

To capture richer semantic relationships in textual data, pre-trained GloVe (Global Vectors for Word Representation) embeddings were used. A separate copy of the PCA-transformed dataset (**df_glove**) was created to preserve the original data.

Pre-trained 100-dimensional GloVe vectors were loaded. These embeddings are trained on large corpora and capture global word co-occurrence patterns, providing stronger semantic representation than locally trained embeddings.

All object-type columns were identified as text features and processed individually. Each text entry was tokenized, and corresponding GloVe vectors were extracted for valid tokens. Row-level representations were obtained by averaging all word vectors. If no valid tokens were found, a zero vector of size 100 was assigned. Columns with a single unique value were skipped.

The resulting embeddings were prefixed by column name (e.g., Product_Name_glove_*) and concatenated with the dataset. After transformation, the dataset size increased to **(2000, 954)**.

While GloVe improves semantic understanding using pre-trained knowledge, it significantly increases dimensionality and computational cost, which may affect training efficiency.

Chapter 5: Machine Learning Modelling - Classification

5.1 Logistic Regression

Multinomial Logistic Regression was used as the baseline model for the 5-class demand classification problem. The target variable, *Daily_Units_Sold*, was transformed into five quantile-based categories using pd.qcut, resulting in a new variable *Demand_Bin* ranging from 0 (lowest demand) to 4 (highest demand).

Bin	Approx. Samples
0 (Lowest)	409
1	400
2	403
3	391
4 (Highest)	397

Table 3: Classification Bins

A classification-based evaluation pipeline was implemented to compare different feature representation techniques for demand prediction. Only numeric features were retained from each vectorized dataset, and the target variable was encoded into demand bins. The dataset was then standardized and split into training, validation, and test sets in a stratified manner to preserve class distribution and ensure fair evaluation. A Logistic Regression model was used as the baseline classifier due to its simplicity and interpretability.

The model was trained and evaluated across multiple vectorization approaches, including categorical encoding, TF-IDF, Word2Vec, and GloVe representations. Performance was measured using accuracy and weighted F1-score on training, validation, and test sets to assess both learning behavior and generalization. This setup enabled a comparative analysis of how different feature engineering strategies impact classification performance, providing a strong baseline for subsequent advanced modeling approaches.

5.2 Diamond Deep Neural Network

A custom feedforward deep neural network with a diamond-shaped architecture was developed for 5-class demand classification. The model expands to capture complex feature interactions and then compresses for refined predictions.

It was trained using the Adam optimizer (learning rate = 0.001), Sparse Categorical Crossentropy loss, for 20 epochs with a batch size of 32.

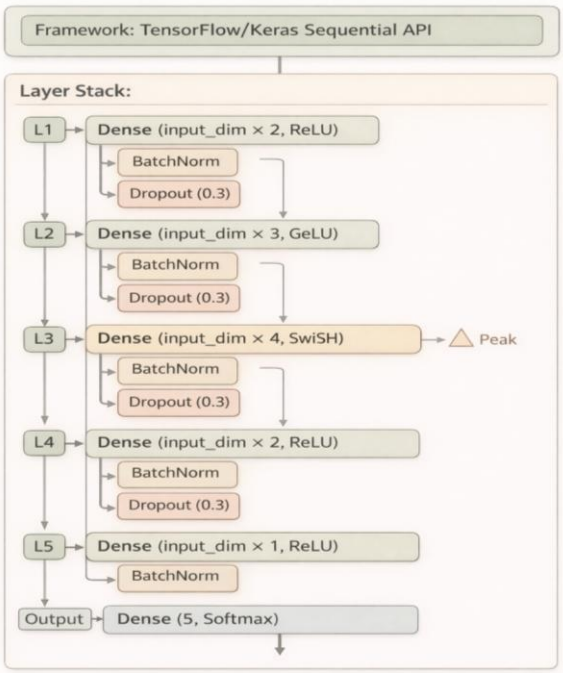

Figure 8: Custom Deep Learning model

A unified deep learning classification model (referred to as the “Diamond Deep Learning Model”) was developed and evaluated across multiple feature vectorization techniques. The input data was standardized and split into training, validation, and test sets using a stratified approach to maintain class balance. This ensured fair evaluation and prevented data leakage. The model was applied consistently across all vectorized representations, including categorical encoding, TF-IDF, Word2Vec, and GloVe embeddings, enabling a direct performance comparison under a common architecture.

The Diamond model follows a deep feedforward neural network design with progressively expanding and contracting hidden layers, incorporating activation functions such as ReLU, GELU, and Swish, along with Batch Normalization and Dropout for improved stability and regularization. The model was trained using the Adam optimizer with sparse categorical cross-entropy loss and

evaluated using accuracy and weighted F1-score on training, validation, and test sets. This setup allowed a comprehensive assessment of how different feature engineering strategies perform under a unified deep learning framework, providing deeper insights into model generalization and robustness.

5.3 Hybrid Transformer Model

A hybrid classification approach combining DistilBERT and XGBoost was implemented to improve demand classification performance. The target variable (daily units sold) was converted into discrete demand bins using quantile-based binning to frame the problem as a multi-class classification task. The input data was preprocessed by handling missing values, converting all features into a consistent string format, and constructing a unified textual representation of each record. This ensured compatibility with transformer-based embedding generation.

DistilBERT was used as a feature extractor (without fine-tuning) to generate contextual embeddings from the textual representation of each data point. These embeddings were then combined with numeric features to form a rich hybrid feature space capturing both semantic and structured information. The final feature set was used to train an XGBoost classifier, which performed the final prediction of demand categories. This combination leverages the strength of deep contextual representation from DistilBERT and the robustness of gradient boosting, resulting in improved classification performance and better handling of complex feature interactions in retail demand data.

Chapter 6: Machine Learning Modelling - Regression

6.1 Random Forest Regressor

A regression-based evaluation pipeline was implemented using a Random Forest Regressor to predict continuous demand values. The dataset was prepared by retaining only numeric features from different vectorized representations, including categorical encoding, TF-IDF, Word2Vec, and GloVe embeddings. The target variable (daily units sold) was sourced from the original dataset. A structured train-validation-test split was applied to ensure fair evaluation and to prevent data leakage, enabling consistent comparison across different feature engineering approaches.

The Random Forest model was trained independently on each feature representation and evaluated using standard regression metrics, including R^2, RMSE, and MAE across training, validation, and test sets. This allowed a comprehensive assessment of both predictive accuracy and generalization performance. The results provided insights into how different vectorization techniques influence regression performance, serving as a strong baseline for understanding demand forecasting behaviour before transitioning to more advanced deep learning and hybrid models.

6.2 Diamond Deep Learning Regression Model

This unified diamond-shaped Deep Neural Network performs regression for precise demand forecasting (Daily_Units_Sold prediction). The architecture systematically expands from input dimensions to capture high-order interactions, then progressively compresses for refined predictions.

Layer	Neurons	Activation	Regularization	Phase
L1	input_dim × 2	ReLU	BatchNorm + Dropout(0.3)	Expansion
L2	input_dim × 3	GELU	BatchNorm + Dropout(0.3)	Expansion
L3	input_dim × 4	Swish	BatchNorm + Dropout(0.3)	Peak Capacity
L4	input_dim × 2	ReLU	BatchNorm + Dropout(0.3)	Compression

L5	input_dim × 1	ReLU	BatchNorm	Compression
Output	1	Linear	-	Prediction

Table 4: Deep Learning model

A unified deep learning regression model (Diamond Deep Learning architecture) was developed to predict continuous demand values across multiple feature vectorization techniques. The dataset was pre-processed by retaining only numerical features from categorical encoding, TF-IDF, Word2Vec, and GloVe representations. The target variable (daily units sold) was added from the original dataset. A structured train-validation-test split was applied to ensure consistent evaluation and prevent data leakage across all experiments.

The Diamond model follows a deep feedforward neural network architecture with progressively expanding and contracting layers, incorporating activation functions such as ReLU, GELU, and Swish, along with Batch Normalization and Dropout for regularization and stability. The model was trained using the Adam optimizer with mean squared error loss and evaluated using MAE, RMSE, and R^2 metrics across training, validation, and test sets. This setup enabled a comprehensive comparison of different feature engineering strategies under a unified deep learning framework, highlighting their impact on predictive accuracy and generalization performance.

6.3 Hybrid Transformer Regression Model

A hybrid forecasting architecture was developed by integrating a pretrained time-series foundation model (Chronos [7]) with a gradient-boosted decision tree model (XGBoost) and a downstream inventory optimization layer. The system is designed to address retail demand forecasting and support inventory decision-making under uncertainty. Unlike traditional transformer models used in natural language processing, this approach leverages a time-series-specific transformer backbone that captures temporal patterns such as seasonality, trends, and non-linear variations directly from data, eliminating the need for extensive manual feature engineering.

The core forecasting component is Amazon Chronos (T5-small variant), a pretrained probabilistic time-series model available on the Hugging Face model hub, trained on large-scale synthetic and real-world datasets. It takes historical demand sequences for each product as input and generates multi-step forecasts of future demand. Chronos is particularly effective in capturing temporal

dynamics such as seasonality, trend shifts, and noise, and performs well even in scenarios with limited data, making it suitable for retail applications with sparse or cold-start conditions.

To enhance prediction accuracy, an XGBoost regressor is incorporated as a residual learning layer. This model is trained on structured tabular features and learns to correct the residual errors of the Chronos forecasts. It captures additional influences such as product category behavior, store-level interactions, and external factors like promotions when available. The final prediction is obtained by combining both model outputs as follows:

$$\hat{y}_{final} = \hat{y}_{Chronos} + \hat{y}_{XGBoost}$$

This hybrid ensemble approach improves overall performance by reducing both bias and variance. Chronos effectively models temporal dependencies (i.e., when demand changes), while XGBoost explains structured relationships and external drivers (i.e., why demand changes), resulting in a more accurate and robust forecasting system.

On top of the forecasting module, a deterministic inventory optimization layer is implemented to convert predictions into actionable supply chain decisions. The key formulations used in this layer are:

- Lead Time Demand: $D_{LT} = \sum_{t=1}^{L} \hat{y}_t$
- Safety Stock: $SS = z \cdot \sigma \cdot \sqrt{L}$
- Reorder Point: $ROP = D_{LT} + SS$
- Reorder Quantity: $Q = max(0, ROP - Stock_{\ on\ hand})$
- Stockout Risk: $Risk = \frac{D_{LT} - Stock}{D_{LT}}$

This layer ensures that forecast outputs are directly translated into inventory policies, enabling efficient stock management while minimizing the risk of stockouts.

Overall, the proposed system offers several advantages, including minimal reliance on manual feature engineering, effective modelling of complex and non-linear demand patterns, and seamless integration of deep learning and machine learning techniques. Additionally, the architecture is scalable across multiple products and provides a direct link between forecasting outputs and operational inventory decisions, making it highly practical for real-world retail applications.

Chapter 7: Results and Interpretation

7.1 Metric Justification

Accuracy is adopted as the primary evaluation metric, given the approximately balanced class distribution achieved through quantile-based binning. To complement this, the weighted F1-score is reported as a secondary metric, as it provides a harmonized measure of precision and recall while accounting for any residual class imbalance. Furthermore, a detailed per-class classification report is included to assess model performance across individual demand categories. For the regression baseline, Root Mean Squared Error (RMSE), R^2 score, and Mean Absolute Error (MAE) are reported. Among these, RMSE is given particular emphasis due to its sensitivity to large errors, making it especially suitable for demand forecasting tasks where significant prediction deviations can lead to disproportionately high operational and financial impact.

Metric	Task	Justification
Accuracy	Classification	Balanced classes; direct business interpretability (% correctly classified demand levels)
Weighted F1	Classification	Balances precision/recall across imbalanced 5 demand bins; penalizes class-specific failures
Precision	Classification	Critical for inventory: penalizes over-prediction (excess stock, carrying costs)
Recall	Classification	Critical for availability: penalizes under-prediction (stockouts, lost sales)
RMSE	Regression	Penalizes large demand errors heavily; same units as target (Daily_Units_Sold)
R^2	Regression	Scale-free variance explained; enables model comparison across datasets
MAE	Regression	Interpretable average error; robust to outliers; direct business translation

Table 5: Evaluation Metrics

7.2 Final Model Selection

While traditional machine learning models such as Logistic Regression and standard deep learning approaches provide strong baseline performance, they are limited in their ability to capture complex temporal dependencies and dynamic demand patterns present in retail time-series data. Logistic Regression is restricted to linear relationships, while standard deep learning models require large datasets and do not inherently model sequential behavior unless explicitly designed for it.

In this project, a hybrid forecasting architecture combining Chronos (a pretrained time-series transformer) and XGBoost (a gradient boosting-based residual learning model) is selected as the final solution due to its ability to effectively capture both temporal and contextual demand patterns.

The Chronos model captures key time-series characteristics such as trend, seasonality, and demand fluctuations, and performs well even with limited historical data due to its pretraining on large-scale datasets. XGBoost complements this by learning structured relationships from tabular features and correcting residual errors left by the base forecast, improving overall accuracy and robustness.

This hybrid approach combines sequence-level forecasting with residual refinement, resulting in a more stable and accurate prediction system. It performs consistently across different SKUs, handles non-linear demand behavior effectively, and is well-suited for real-world retail forecasting scenarios.

Therefore, the Chronos + XGBoost hybrid model is selected as the final model because it provides superior predictive performance, better generalization, and improved suitability for downstream inventory optimization compared to standalone models.

7.3 Model Deployment and Inference Pipeline

To enable real-time predictions, the final trained model was deployed using **FastAPI**, a high-performance Python framework for building APIs with automatic documentation support.

The deployment pipeline loads the trained model and preprocessing components, receives input data through API endpoints, applies the same preprocessing steps, and returns predictions in real time. Endpoints were created for both health checks and prediction requests, allowing users to interact with the model using structured inputs.

FastAPI also provides interactive documentation via Swagger UI, making it easy to test and validate the API. This deployment approach ensures scalability, reproducibility, and easy integration of the model into real-world applications.

Key Features of Deployment:

- Real-time prediction capability
- Consistent preprocessing using saved pipelines
- Lightweight and scalable architecture
- Easy integration with frontend or external systems

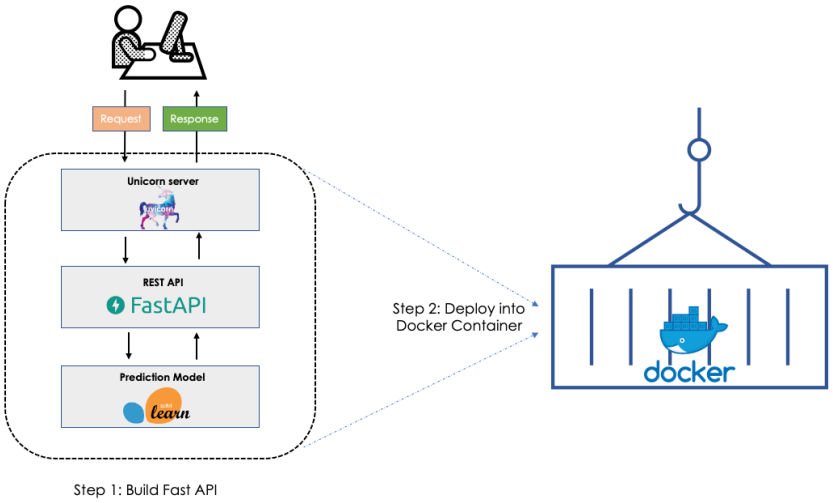

Figure 9: FastAPI architecture

The deployment system provides real-time demand prediction through a lightweight and scalable FastAPI service. User inputs are received via the API and validated using a defined schema to ensure correctness and completeness.

Validated inputs are passed through the same preprocessing pipeline used during training to maintain consistency between training and inference. The processed data is then fed into the trained model to generate the predicted demand class.

The final prediction is returned in JSON format, enabling easy integration with external systems such as dashboards or web applications. A health check endpoint is also included to monitor API availability and system status.

Chapter 8: Code Implementation and Execution Results

This section presents the implementation of the proposed retail demand forecasting system, including key code snippets and corresponding outputs to demonstrate the practical execution of the pipeline. This section highlights the successful execution of the system and demonstrates the reproducibility of the proposed solution.

8.1 Data Cleaning and Preprocessing

This step involves transforming the raw dataset into a clean and consistent format by handling missing values, converting date features, encoding binary variables, and removing high-cardinality columns.

```
After Cleaning Shape: (2000, 58)

Remaining Object Columns:
 ['Category', 'Store_Type', 'Festival_Name', 'Festival_Type', 'Product_Name', 'Sub_Category', 'Brand_Name', 'Brand_Tier', 'Promotion_Type']
```

Figure 10: Data Preprocessing output

8.2 Feature and Target Definition

In this step, the dataset is divided into input features and the target variable, followed by dynamic identification of numerical, categorical, and text features.

```
Target Column: Daily_Units_Sold
Numerical Columns: ['DayOfWeek', 'Month', 'Year', 'Is_Weekend', 'Is_Holiday', 'Product_ID', 'Discount_Percentage', 'Store_ID', 'Footfall_Index', 'Avg_Tem
perature', 'Rainfall_mm', 'Social_Media_Sentiment', 'Lead_Time_Days', 'Safety_Stock_Level', 'Stock_On_Hand', 'UPC_EAN', 'Shelf_Life_Days', 'No_of_Checkou
t_Counters', 'Avg_Billing_Time_min', 'Local_Population_Density', 'Product_Age_Days', 'No_of_Customer_Purchases', 'Promotional_Campaign_Flag', 'Competitor
_Promotion_Flag', 'Google_Trends_Current_Wk', 'Google_Trends_Lag_1w', 'Shelf_Capacity', 'Marketing_Spend', 'Payday_Flag', 'School_Vacation_Flag', 'Local_
Event_Flag', 'Seasonal_Product_Flag', 'Online_Sales_Units', 'In_Store_Sales_Units', 'App_Traffic_Index', 'Website_Visits', 'Backorder_Flag', 'Supplier_De
lay_Days', 'Fill_Rate_Pct', 'Loyalty_Program_Usage_Count', 'Repeat_Purchase_Rate', 'Avg_Basket_Size']
Categorical Columns: ['Category', 'Store_Type', 'Festival_Type', 'Sub_Category', 'Brand_Name', 'Brand_Tier', 'Promotion_Type']
Text Columns: ['Festival_Name', 'Product_Name']
```

Figure 11: Feature Split output

8.3 Feature Preprocessing and Scaling

This stage includes handling missing values, removing ID-like columns, and applying standardization to numerical features to prepare the dataset for modelling.

```
Processed Shape (ready for vectorization later): (2000, 56)
Numeric columns: ['DayOfWeek', 'Month', 'Year', 'Is_Weekend', 'Is_Holiday', 'Product_ID', 'Discount_Percentage', 'Store_ID', 'Footfall_Index', 'Avg_Tempe
rature', 'Rainfall_mm', 'Social_Media_Sentiment', 'Lead_Time_Days', 'Safety_Stock_Level', 'Stock_On_Hand', 'UPC_EAN', 'Shelf_Life_Days', 'No_of_Checkout_
Counters', 'Avg_Billing_Time_min', 'Local_Population_Density', 'Product_Age_Days', 'No_of_Customer_Purchases', 'Promotional_Campaign_Flag', 'Competitor_P
romotion_Flag', 'Google_Trends_Current_Wk', 'Google_Trends_Lag_1w', 'Shelf_Capacity', 'Payday_Flag', 'School_Vacation_Flag', 'Local_Event_Flag', 'Seasona
l_Product_Flag', 'Online_Sales_Units', 'In_Store_Sales_Units', 'App_Traffic_Index', 'Website_Visits', 'Backorder_Flag', 'Supplier_Delay_Days', 'Fill_Rate
_Pct', 'Loyalty_Program_Usage_Count', 'Repeat_Purchase_Rate', 'Avg_Basket_Size']
Categorical columns (kept as-is): ['Category', 'Store_Type', 'Festival_Type', 'Sub_Category', 'Brand_Name', 'Brand_Tier', 'Promotion_Type']
Text columns (kept as-is): ['Festival_Name', 'Product_Name']
```

Figure 12: Feature preparation output

8.4 Dimensionality Reduction using PCA

Principal Component Analysis (PCA) is applied to highly correlated features to reduce redundancy while retaining maximum variance in the dataset.

```
Features selected for PCA: ['Online_Sales_Units', 'Month', 'Website_Visits', 'Date_month', 'App_Traffic_Index']
PCA explained variance ratios: [0.60136944 0.39862889]
Cumulative variance: 0.9999983298659787
New shape after PCA: (2000, 54)
```

Figure 13: PCA output

8.5 Categorical Feature Encoding

Categorical features are transformed into numerical representations using label encoding to ensure compatibility with machine learning algorithms.

```
Label encoding complete.
Categorical columns encoded: ['Category', 'Store_Type', 'Festival_Type', 'Sub_Category', 'Brand_Name', 'Brand_Tier', 'Promotion_Type']
Text columns (kept as-is): ['Festival_Name', 'Product_Name']
Shape after encoding: (2000, 54)
```

Figure 14: Encoding output

8.6 TF-IDF Vectorization

Textual features are converted into numerical form using TF-IDF to capture the importance of terms across the dataset.

```
TF-IDF vectorization complete.
Text columns vectorized: ['Category', 'Store_Type', 'Festival_Name', 'Festival_Type', 'Product_Name', 'Sub_Category', 'Brand_Name', 'Brand_Tier', 'Promot
ion_Type']
New shape: (2000, 192)
```

Figure 15: TF - IDF output

8.7 Word2Vec Embedding

Word2Vec is used to generate dense vector representations of text data by capturing semantic relationships between words.

```
Word2Vec embeddings added.
New shape: (2000, 504)
```

Figure 16: Word2Vec output

8.8 GloVe Embedding

Pre-trained GloVe embeddings are utilized to incorporate global semantic knowledge into text feature representation

```
GloVe embedding: 100%|██████████| 9/9 [00:02<00:00,  4.04it/s]
GloVe embeddings added.
New shape: (2000, 954)
```

Figure 17: GloVe output

8.9 Model Training – Logistic Regression (Classification Baseline)

A baseline Logistic Regression model is trained to establish a reference point for evaluating more complex models.

```
===== TRAIN RESULTS =====
```

TRAIN RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.958889	0.958904
1	TF-IDF	0.983333	0.983333
2	Word2Vec	0.983333	0.983328
3	GloVe	0.971111	0.971122

```
===== VALIDATION RESULTS =====
```

VALIDATION RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.920000	0.918665
1	TF-IDF	0.870000	0.869558
2	Word2Vec	0.880000	0.879059
3	GloVe	0.940000	0.939191

```
===== TEST RESULTS =====
```

TEST RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.940000	0.940488
1	TF-IDF	0.860000	0.862918
2	Word2Vec	0.850000	0.853086
3	GloVe	0.900000	0.901076

Figure 18: Logistic Regression output

8.10 Model Training – Deep Learning Model (Classification)

A custom deep learning model is developed to capture complex nonlinear relationships within the data.

```
===== TRAIN RESULTS =====
```

TRAIN RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.953333	0.953100
1	TF-IDF	0.985000	0.984986
2	Word2Vec	0.929444	0.928882
3	GloVe	0.890556	0.890287

```
===== VALIDATION RESULTS =====
```

VALIDATION RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.840000	0.836675
1	TF-IDF	0.760000	0.757503
2	Word2Vec	0.770000	0.768765
3	GloVe	0.800000	0.793669

```
===== TEST RESULTS =====
```

TEST RESULTS

	Model	Accuracy	F1
0	Categorical Encoding	0.920000	0.919679
1	TF-IDF	0.780000	0.780887
2	Word2Vec	0.740000	0.735370
3	GloVe	0.780000	0.776674

Figure 19: Deep Learning output

8.11 Model Training – Transformer Model (Classification)

A pre-trained DistilBERT-based transformer is used as a frozen feature extractor to capture contextual representations, which are combined with XGBoost for improved classification performance.

```
===== DISTILBERT + XGBOOST RESULTS =====
Accuracy: 0.92
              precision    recall  f1-score   support

           0       0.95      0.96      0.96        82
           1       0.91      0.93      0.92        80
           2       0.94      0.81      0.87        81
           3       0.84      0.94      0.88        78
           4       0.96      0.96      0.96        79

    accuracy                           0.92       400
   macro avg       0.92      0.92      0.92       400
weighted avg       0.92      0.92      0.92       400
```

Figure 20: Transformer output

8.12 Model Training – Random Forest (Regression Baseline)

A baseline Regression model is trained to establish a reference point for evaluating more complex models.

```
===== TRAIN RESULTS =====
```

TRAIN RESULTS

	Model	R2	RMSE	MAE
0	Categorical Encoding	0.999	0.70	0.42
1	TF-IDF	0.999	0.71	0.42
2	Word2Vec	0.999	0.77	0.47
3	GloVe	0.999	0.75	0.45

```
===== VALIDATION RESULTS =====
```

VALIDATION RESULTS

	Model	R2	RMSE	MAE
0	Categorical Encoding	0.998	1.31	0.99
1	TF-IDF	0.998	1.36	1.03
2	Word2Vec	0.997	1.55	1.17
3	GloVe	0.997	1.45	1.09

```
===== TEST RESULTS =====
```

TEST RESULTS

	Model	R2	RMSE	MAE
0	Categorical Encoding	0.998	1.32	1.00
1	TF-IDF	0.998	1.32	1.00
2	Word2Vec	0.997	1.44	1.11
3	GloVe	0.997	1.38	1.06

Figure 21: Regression output

8.13 Model Training – Deep Learning Model (Regression)

A custom deep learning model is developed to capture complex nonlinear relationships within the data.

```
===== TRAIN RESULTS =====
```

TRAIN RESULTS

	Model	MAE	RMSE	R2
0	Categorical Encoding	17.37	17.51	0.628
1	TF-IDF	3.70	4.61	0.974
2	Word2Vec	4.08	5.41	0.965
3	GloVe	4.36	6.91	0.942

```
===== VALIDATION RESULTS =====
```

VALIDATION RESULTS

	Model	MAE	RMSE	R2
0	Categorical Encoding	17.41	17.61	0.590
1	TF-IDF	3.76	4.72	0.971
2	Word2Vec	5.42	7.62	0.923
3	GloVe	4.58	8.16	0.912

```
===== TEST RESULTS =====
```

TEST RESULTS

	Model	MAE	RMSE	R2
0	Categorical Encoding	17.50	17.69	0.562
1	TF-IDF	3.74	4.51	0.972
2	Word2Vec	5.65	8.63	0.896
3	GloVe	4.80	7.89	0.913

Figure 22: Deep Learning output

8.14 Model Training – Transformer Model (Regression)

A pre-trained Chronos time-series transformer is used to generate demand forecasts, which are enhanced using XGBoost residual learning and further integrated with an inventory optimization layer for improved decision-making.

```
===== FORECAST RESULTS =====

    Product_ID        MAE       RMSE        R2
0         1000  12.312825  15.022709  0.618176
1         1001   3.012894   3.456274  0.956726
2         1002  18.611513  20.398903  0.345329
3         1003   4.649886   5.730253  0.966867
4         1004   4.711232   6.501054  0.927548
5         1005   4.060368   5.062129  0.915610
6         1006   7.810316   9.151858  0.915487
7         1007   4.932600   6.713631  0.935278
8         1008   8.921687   9.963290  0.870858
9         1009   5.983162   7.596968  0.803721
10        1010   9.549717  11.550968  0.786082
11        1011  10.898387  13.174162  0.896507
12        1012  11.118055  12.212884  0.804734
13        1013   9.735107  11.018876  0.772073
14        1014  16.826440  19.567263  0.230396
15        1015   7.247684   9.235979  0.840173
16        1016   6.326161   8.672742  0.880781
17        1017  13.118926  14.350916  0.772364
18        1018   7.303127  10.088056  0.875430
19        1019  10.264830  11.704533  0.895063
Product_ID    1009.500000
MAE              8.869746
RMSE            10.558672
R2               0.800460
dtype: float64
```

Figure 23: Transformer output

```
===== INVENTORY OPTIMIZATION RESULTS =====

    lead_time_demand  safety_stock  reorder_point  stockout_risk  reorder_qty  Product_ID
0         879.659485     39.028970     918.688455       0.483891   464.688455        1000
1         768.576904      7.847073     776.423978       0.612270   478.423978        1001
2         676.526428     18.154468     694.680897       0.382138   276.680897        1002
3         786.179504      9.749282     795.928787       0.795212   634.928787        1003
4         780.357849     16.743800     797.101649       0.639909   516.101649        1004
5         800.253662     14.129236     814.382898       0.446426   371.382898        1005
6         758.031250     20.784880     778.816130       0.646453   510.816130        1006
7         803.204834     12.234458     815.439292       0.815738   667.439292        1007
8         775.928467     17.683635     793.612102       0.609500   490.612102        1008
9         804.401917     12.939086     817.341003       0.658131   542.341003        1009
10        835.728821     14.927740     850.656561       0.717612   614.656561        1010
11        868.151978      8.509947     876.661924       0.623338   549.661924        1011
12        884.690796     13.649921     898.340717       0.706112   638.340717        1012
13        793.305298     17.250392     810.555690       0.649567   532.555690        1013
14        949.246216     13.945708     963.191923       0.491175   480.191923        1014
15        720.431213     18.393924     738.825138       0.350389   270.825138        1015
16        767.671021     15.726158     783.397179       0.351285   285.397179        1016
17        701.424255     17.448625     718.872880       0.295719   224.872880        1017
18        743.773865     13.968898     757.742763       0.608752   466.742763        1018
19        771.743103     17.788910     789.532013       0.968902   765.532013        1019
```

Figure 24: Transformer output

8.15 Fast API

Model Summary

- **Name**: UnifiedDiamondDL
- **Purpose**: Retail demand forecasting
- **Input**: 18 named features (prices, sales, promotion, store, date, social)
- **Output**: Daily units sold (1 continuous value)
- **Type**: Deep Learning Regression with diamond architecture
- **Framework**: TensorFlow/Keras

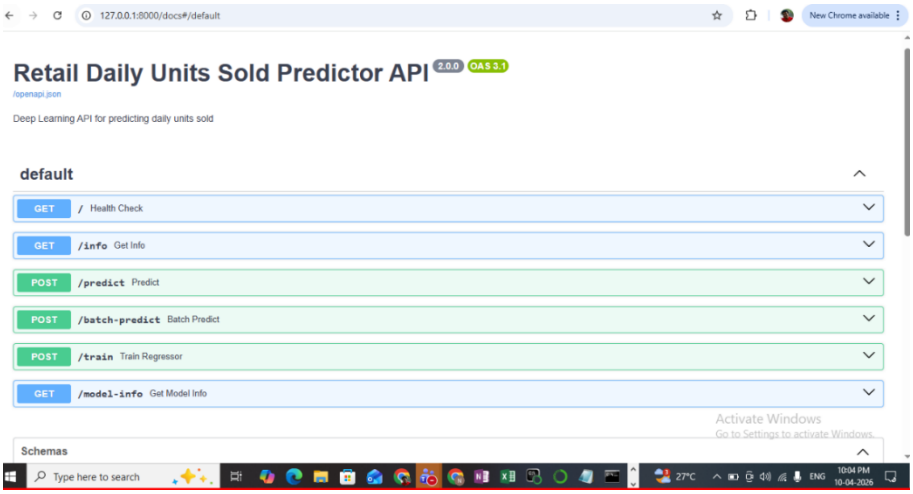

Figure 25: FastAPI default UI

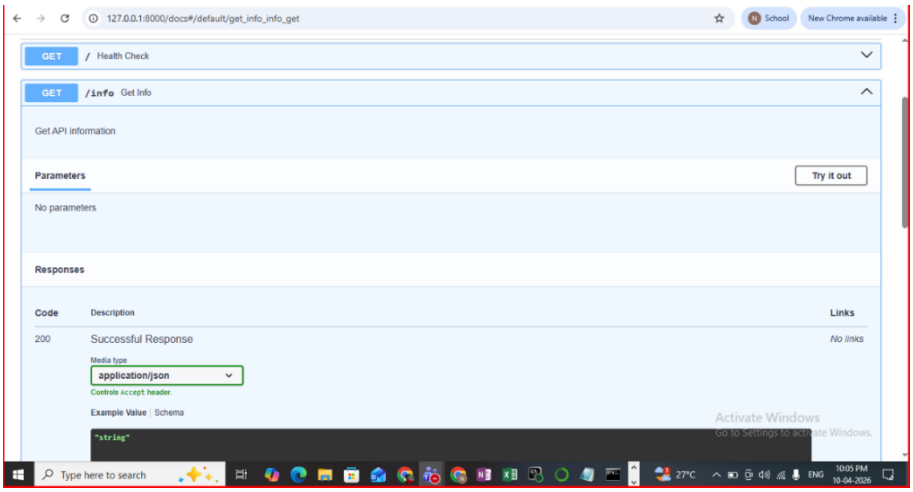

Figure 26: FastAPI parameters view

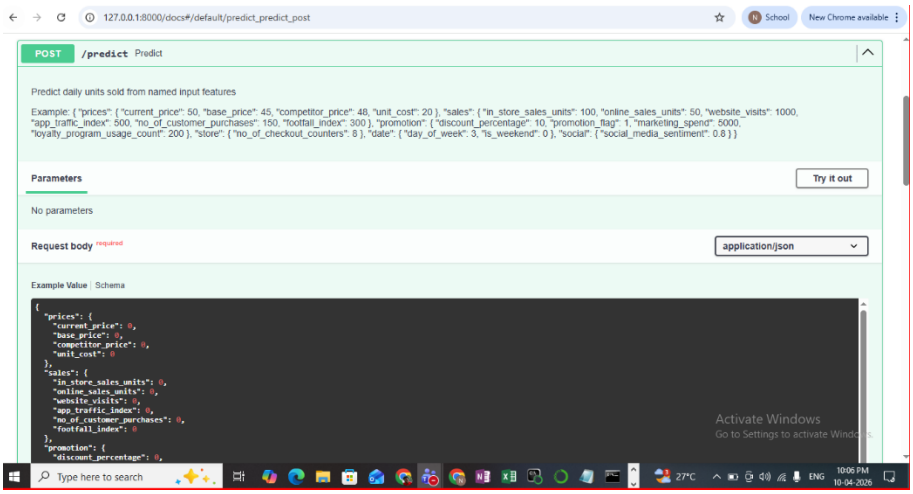

Figure 27: FastAPI predict view

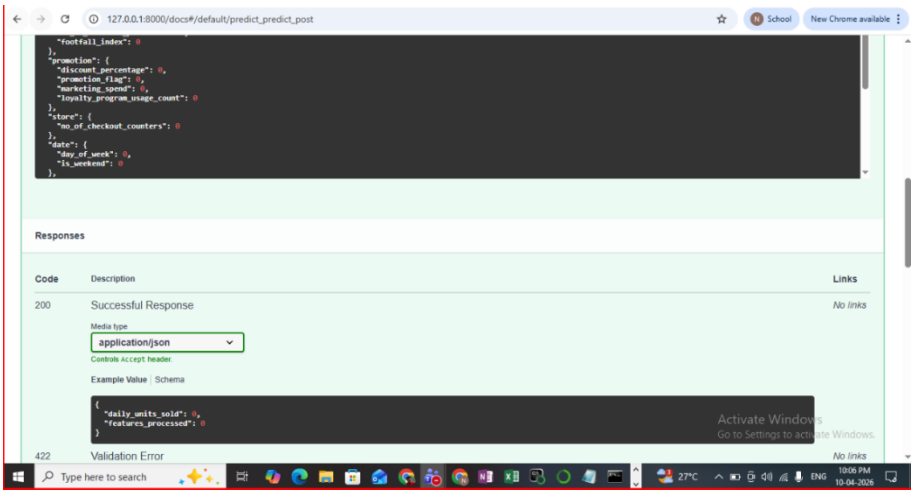

Figure 28: FastAPI responses

Chapter 9: Conclusions and Future Work

9.1 Conclusion

This project addressed the problem of retail demand forecasting and inventory optimization by developing an end-to-end machine learning pipeline that integrates classical machine learning, deep learning, and transformer-based time-series modeling. By framing the problem using both regression and classification perspectives, the study enabled a structured evaluation of multiple predictive approaches. The pipeline included data preprocessing, feature engineering, categorical encoding, and time-aware train-test splitting, ensuring robust and leakage-free model evaluation.

Baseline models such as Logistic Regression and deep learning architectures provided useful benchmarks but were limited in capturing the sequential and non-linear nature of retail demand data. To overcome these limitations, a hybrid forecasting system was developed by combining Chronos, a pretrained time-series transformer, with XGBoost as a residual learning model. Chronos captures temporal patterns such as trend, seasonality, and irregular fluctuations, while XGBoost improves accuracy by correcting residual errors and modeling structured relationships. This combination enhances both stability and predictive performance.

The system is further extended with an inventory optimization layer that converts demand forecasts into actionable decisions. Using standard supply chain formulations such as safety stock, reorder point, reorder quantity, and stockout risk, the model enables efficient inventory management under uncertainty. This transforms the solution from a predictive model into a practical decision-support system for retail operations.

Additionally, a deep learning model developed in this project was deployed using FastAPI, enabling it to function as a scalable API for demand prediction. This allows real-time or batch inference through a lightweight web framework, demonstrating the practical usability and deployment readiness of the model.

In conclusion, this project demonstrates that combining transformer-based foundation models with gradient boosting techniques, along with optimization logic and deployment via FastAPI, provides a scalable and effective solution for retail demand forecasting. The integration of prediction, decision-making, and deployment highlights the importance of building end-to-end, production-oriented machine learning systems.

9.2 Directions for Future Work

- **Advanced Time-Series Architectures:** Explore models such as Temporal Fusion Transformers (TFT), Informer, and N-BEATS to further improve long-range dependency learning and multi-horizon forecasting performance beyond the current Chronos-based system.
- **Probabilistic Forecasting:** Extend the current deterministic forecasting approach to probabilistic outputs to generate prediction intervals, enabling uncertainty-aware decision-making for inventory planning and safety stock estimation.
- **Improved Residual Learning:** Enhance the hybrid framework by experimenting with stacked residual learners or neural network-based residual correction models to better capture complex non-linear error patterns beyond XGBoost.
- **Advanced Feature Engineering:** Incorporate lag features, rolling statistics, seasonality indicators, and demand volatility measures to improve model performance and interpretability.
- **External Data Integration:** Include contextual signals such as promotions, holidays, weather data, and market events to improve forecasting accuracy by capturing real-world demand influences.
- **Multi-Model Ensemble Systems:** Extend the current hybrid approach into a broader ensemble combining transformer models, gradient boosting, and lightweight deep learning models for improved robustness and stability across SKUs.
- **Dynamic Inventory Optimization:** Develop adaptive or reinforcement learning-based inventory policies that continuously adjust reorder points and quantities based on changing demand patterns and uncertainty.
- **Scalability and Cloud Deployment:** Extend the FastAPI-based deployment to cloud platforms such as AWS, Azure, or Google Cloud to enable large-scale, production-ready forecasting with automated retraining and monitoring pipelines.

References

[1] Oracle, “AI Demand Forecasting in Supply Chain Management,” Oracle, [Online]. Available: https://www.oracle.com/scm/ai-demand-forecasting/. [Accessed 2026].

[2] IBM, “What is demand forecasting?,” IBM, [Online]. Available: https://www.ibm.com/topics/demand-forecasting. [Accessed 2026].

[3] M. I. o. T. (MIT), “Advanced AI/ML for Demand Forecasting,” Massachusetts Institute of Technology (MIT), [Online]. Available: https://digitalsc.mit.edu/advanced-aiml-demand-forecasting/. [Accessed 2026].

[4] P. S. D. C. P. R. S. B. a. A. M. S. Dhage, “Demand Forecasting with Machine Learning: A Data-Driven Strategy for Market Prediction,” in *Proc. IEEE Pune Section Int. Conf. (PuneCon)*, Pune, 2025.

[5] K. &. G. D. &. D. S. &. S. Y. Poongothai, “Smart Retail Using Machine Learning for Demand Forecasting and Inventory Optimization,” in *Proc. Int. Conf. Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*, 2024.

[6] R. R. K. M. A. M. V. L. B. A. a. P. M. D. D. Winster Praveenraj, “Optimizing Retail Operations with Big Data-Driven Insights: From Inventory Management to Personalized Marketing,” in *International Conference on Automation and Computation (AUTOCOM)*, Dehradun, India, 2025.

[7] Amazon, “Chronos: Learning the Language of Time Series,” Amazon, [Online]. Available: https://arxiv.org/abs/2403.07815. [Accessed April 2026].